

SAP ABAP 整合與介面開發課程 (BAPI / Data Conversion / RFC / DBCO / JDBC)

REST 課程 ([src/ABAP_Training_REST/](#)) rs10/rs11 已經帶學員呼叫過 `BAPI_FLBOOKING_CREATEFROMDATA` 與 `BAPI_TRANSACTION_COMMIT`，但只教了「怎麼用」，沒有系統性講「BAPI 是什麼、跟一般 Function Module 差在哪、怎麼自己找到需要的 BAPI」；本課程補齊這條「跟外部系統 / 資料交換」的整合技術線，並往下延伸到更底層的 RFC Native SQL、DBCO、JDBC 連線。**if01 ~ if09 全數完成並在系統上驗證，主課程結案 (2026-07-21)。**

課程定位

- 對象：完成基礎課 (尤其 ex15 Function Module、ex23 LUW all-or-nothing) 的學員；建議已上過 REST 課 rs10/rs11 (有呼叫 BAPI 的第一手經驗)，但非必要前提。
- 技術範圍：BAPI 觀念與尋找方法、Data Conversion (資料轉換 / 搬遷) 常見技術比較與 **BDC Program** 實作、RFC 基礎與 Native SQL、DBCO 與 RFC Destination 設定、ADBC (現代 JDBC 風格連線 API)。不含實際 IDoc / PI-PO / CPI 等中介系統的介接 (那是另一條更大的整合技術線，列為下一階段候選)。
- 已知工具限制 (**if07 已實測確認**)：SM59 (RFC Destination)、DBCO (Database Connection) 確認沒有 ADT API ([discovery](#) 全文查無相關 collection)，屬於本專案已知的 GUI-only 類別；底層表 [RFCDES](#) 可用 Open SQL / [datapreview/freestyle](#) 正常讀取，[DBCON](#) 則被 [datapreview/freestyle](#) 這個特定工具擋下 (但 if08 證實 [DBCON](#) 本身完全可被 Open SQL 正常讀取，擋讀只在該工具層級)，實際連線設定仍需使用者在 SAP GUI / DBACOCKPIT 完成。
- 結業標準 (草案)：能分辨一個介面該叫 FM 還是 BAPI、知道去哪裡找需要的標準 BAPI、能寫出正確的 BAPI 呼叫 + `COMMIT/ROLLBACK` 序列、理解 Data Conversion 幾種主流做法的取舍、能用 **SHDB** 錄製並寫出一支可執行的 **BDC** 程式 (**Call Transaction** 與 **Session** 兩種方式都要會)、看得懂 Native SQL 與 ADBC 語法並知道使用時機與風險。

教材慣例 (比照 OOP/REST/AMDP 課程)

- 每題三件套：題目 [ifnn_主題.md](#) + PDF 講義 ([node tools/md2pdf.js src/ABAP_Training_Interface](#)) + 答案快照
- 每題 md 開頭 (**## 學習目標** 之前) 要有 **## Lecture** 完整背景知識講解
- 答案物件命名：沿用 `ZCL_IFnn_*` (Service 類別) / `ZR_IFnn_*` (呼叫端程式)
- 資料模型：優先沿用 SCARR/SFLIGHT/SBOOK 航班模型，跟 REST 課 rs10/rs11 已經教過的 BAPI 情境銜接；例外是 **if02/if03**——「怎麼找 BAPI」這個技能本身要練在**標準模組的真實物件**上才有意義 (航班模型是課程自訂的教學情境，SAP 標準系統不會真的有 `BAPI_FLBOOKING_*` 讓人拿去找)，因此這兩題改用 QM / MM 標準物件 ([BUS2045](#) 檢驗批、[QALS](#) / [QAMB](#) 標準表、`BAPI_INSPLLOT_*` / `BAPI_GOODSMVT_*` 標準 BAPI)，if04 起才轉回沿用航班模型或自訂情境

課綱 (草案，待確認與逐題出題)

#	主題	內容重點	銜接前面課程	狀態
if01	Function Module vs BAPI	BAPI 本質上是「掛在 BOR (Business Object Repository) 下、有版本相容承諾、可被外部系統呼叫」的一種特殊公開 FM；對照表：命名慣例 (<code>BAPI_<物件>_<動作></code>) / RFC-enabled / Release 承諾 (SAP 不隨意改介面) / RETURN 結構化錯誤 vs 一般 FM 的 EXCEPTIONS 。回頭複習 rs10 用過的 <code>BAPI_FLBOOKING_CREATEFROMDATA</code> ，這次講清楚「它為什麼是 BAPI 不是普通 FM」；不需新建物件，沿用 ex15 <code>Z_TR15_CALC_REVENUE</code> 與 rs10 的 BAPI 呼叫端程式做比較練習	承基礎課 ex15 Function Module、REST rs10	已完成 (if01_bapi_vs_fm.md)；已用 <code>sap-adt</code> MCP 連線 client 130 查證 <code>Z_TR15_CALC_REVENUE</code> 非 Remote-Enabled、 <code>BAPI_FLBOOKING_CREATEFROMDATA</code> 是 <code>processingType=rfc + releaseState=external</code> ；唯一查不到的「掛在哪個 BOR Business Object」屬已知 GUI-only 限制)
if02	怎麼找 BAPI (實案：QA12 檢驗批過帳)	以 QA12 (變更使用決策 / Usage Decision) 為題目破口，示範完整尋找流程，並拆成兩種情境對照 (這正是這題最重要的教學眉角)：① 情境 A ——下使用決策的「同時」要自動過帳：交易碼背後的 Business Object 是 BUS2045 (Inspection Lot) ，用 BAPI Explorer / SW01 / SE37 搜尋	承 if01；呼應 REST rs10 的 <code>BAPI_FLBOOKING_CREATEFROMDATA</code>	已完成 (if02_find_bapi_qa12.md)；八支 FM / BAPI 全數已用 <code>sap-adt</code> MCP 於 client 130 查證存在並讀出真實介面簽章，含 <code>BAPI_INSPLLOT_SETUSAGEDECISION</code> 原始碼裡的 COMMIT/ROLLBACK 註解、 <code>QAMB_COLLECT_RECORD</code> 的 <code>PERFORM ... ON COMMIT</code> 延遲寫入機制；只設計不執行，未對任何真實檢驗批做過帳測試)

#	主題	內容重點	銜接前面課程	狀態
		BAPI_INSPLLOT_* 可以找到 BAPI_INSPLLOT_SETUSAGEDECISION (IMPORTING number + ud_data TYPE BAPI2045UD · ud_data- ud_stock_posting = 'X' 觸發自動過帳) · 這支「已判定 + 已過帳」一次做完 · 只適用這種同時發生的情境；②情境 B——單純幫已完成判定的檢驗批補過帳 (對照 SAP 標準的「Post Open Stocks」情境：使用決策已下但未過帳 · 庫存還卡在 Q 狀態) · 這時不能用 BAPI_INSPLLOT_SETUSAGEDECISION · 要自己組三支底層 FM 的呼叫序列： MB_CREATE_GOODS_MOVEMENT + MB_POST_GOODS_MOVEMENT (MIGO 底層實際過帳用的兩階段 FM · CREATE 先在記憶體組憑證、POST 才真正寫入資料庫 · 比 BAPI_GOODSMVT_CREATE 更貼近畫面操作邏輯) → QAMB_COLLECT_RECORD (把剛過帳出來的物料憑證號碼登記進標準表 QAMB · 建立與檢驗批的關聯) → STATUS_CHANGE_INTERN (透過 SAP Status Management 更新檢驗批的系統狀態 · 例如標記為已過帳) ；③這三支底層 FM 不在任何公開 BAPI 清單裡 (QAMB_COLLECT_RECORD / STATUS_CHANGE_INTERN 都不是 Released BAPI · BAPI Explorer / SWO1 / SE37 命名規則搜尋都查不到) · 這題真正的教學核心是「當目標找不到公開 BAPI 時 · 在 SAP 系統內怎麼從『標準系統已經在做這件事』這個事實往回追出實際呼叫序列」 · 依實用性排序教六招：(a) 先找有沒有現成畫面 / 程式在做同一件事——QA12 使用決策畫面本身就有「過帳未過帳數量」的後續動作 · 定位到對應程式後直接在 SE38/SE80 開原始碼 · Ctrl+F 搜尋 CALL FUNCTION 字串 · 靜態就能看到完整呼叫清單 · 比從頭 debug 快；(b) 找不到對應程式 (藏在 BAdI / Enhancement 裡) 時改用 SAT / SE30 (Runtime Analysis)：啟動量測 → 在 QA12 實際執行過帳動作 → 停止量測 · 會產生完整呼叫階層 · 直接篩選 MB_* / QAMB* / STATUS_CHANGE* 關鍵字就能看到本次操作呼叫過的每個 FM / Method 與正確順序；(c) ABAP Debugger 手動追：操作前打 /h 開 debugger · 設 Breakpoint at Statement = CALL FUNCTION (或針對特定欄位設 Watchpoint) · F5/F6 逐步走看 Call Stack · 適合鎖定範圍後的細部確認 · 操作量大不適合一開始就用；(d) ST05 SQL Trace 反查：若切入點是「知道結果會寫進 QAMB 這張		

#	主題	內容重點	銜接前面課程	狀態
		表，但不知道是誰寫的」，開 ST05 錄 SQL Trace、執行過帳動作、在結果找 QAMB 的 INSERT/UPDATE，展開紀錄可直接看到是哪個 Program/Include/行號送出的，從結果反推呼叫者最直接；(e) SE37 依命名慣例輔助猜測（SAP 內部常見「表名_動詞」命名法，搜 QAMB* 有機會撈到候選，但不保證正確，仍要靠 (a)~(d) 驗證）；(f) OSS Notes / SAP Community 搜尋——這正是很多人（包含這次的知識來源）實際上「看別人程式知道要用哪些 FM」的管道，善用關鍵字搜尋常比自己從頭追快；但要注意工具邊界：官方 OSS Notes（launchpad.support.sap.com）需要 S-user 登入，AI 助理查不到，這步驟實務上要靠自己的帳號查，公開的 SAP Help / SAP Community 用一般關鍵字搜尋這兩支底層 FM 名稱通常也查不到直接命中（實測過，這兩支太底層、不是公開文件會收錄的等級），別預期搜得到；但用同樣關鍵字換個角度搜「Post Open Stocks」，能在 SAP Help 查到兩篇官方文件印證情境 B 的存在與確切操作路徑——Post Open Stocks (QM-IM-UD)（說明「UD 下了但沒過帳，貨卡在檢驗庫存，系統甚至會觸發 Workflow 提醒負責人補過帳」正是這個情境）與 After You Have Made a Usage Decision（給出確切標準路徑：QA12 = Usage Decision → Change with History，切到「Inspection lot stock」頁籤輸入要過帳的數量、Save——這個 Save 動作背後呼叫的很可能就是這三支底層 FM，這比盲搜 Note 號碼更好用，直接對應招式 (a)：先鎖定畫面 / 程式再讀原始碼；④反冲對照：MM 直接下庫存異動（不經過 QM）用泛用的 BAPI_GOODSMVT_CREATE + BAPI_GOODSMVT_CANCEL。整組物件名稱由使用者提供並經確認，BAPI_INSLOT_SETUSAGEDECISION / BAPI_INSLOT_GETDETAIL / MB_CREATE_GOODS_MOVEMENT / BAPI_GOODSMVT_CREATE / BAPI_GOODSMVT_CANCEL 已用 sap-docs 交叉查證存在，QAMB_COLLECT_RECORD / STATUS_CHANGE_INTERN / MB_POST_GOODS_MOVEMENT 屬於底層內部 FM（公開文件庫查不到，正常現象），出題時務必在 SAP 系統用 SE37 實際讀一次介面定義、並用 where-used 反查標準程式確認呼叫序列，不能只憑這份草案的描述直接寫程式		

#	主題	內容重點	銜接前面課程	狀態
if03	BAPI 的 COMMIT / ROLLBACK 與 LUW 控制 (延續 QA12 案例)	延續 if02 兩種情境的呼叫序列・複習 rs10 已教的 BAPI_TRANSACTION_COMMIT / BAPI_TRANSACTION_ROLLBACK・這次講透：為什麼 BAPI_INSPLOT_SETUSAGEDECISION / BAPI_GOODSMVT_CREATE 都不做 COMMIT WORK (LUW 邊界交給呼叫端決定) ；情境 B 的三支底層 FM 序列 (MB_CREATE_GOODS_MOVEMENT → MB_POST_GOODS_MOVEMENT → QAMB_COLLECT_RECORD → STATUS_CHANGE_INTERN) 更是必須全部包在同一個 LUW、最後才一次 COMMIT WORK 的典型案例——只要中間任一步失敗就整串 ROLLBACK・避免出現「物料憑證過帳成功、但 QAMB 沒登記關聯、檢驗批狀態也沒更新」這種資料不一致的半套結果；WAIT 參數的意義 (等待非同步 update 完成・確保 COMMIT 返回時 QAMB / 物料憑證真的已落地才能接著查詢) ；對照基礎課 ex23 教過的 ROLLBACK WORK all-or-nothing・比較兩者適用場景的差異	承 if02、基礎課 ex23、REST rs10/rs11	已完成 (if03_commit_rollback_luw.md ；已查證 BAPI_TRANSACTION_COMMIT / ROLLBACK 原始碼・證實兩者就是 COMMIT WORK [AND WAIT] / ROLLBACK WORK 包一層 RETURN・並非獨立機制；WAIT 參數效果對照 if02 的 QAMB_COLLECT_RECORD 延遲寫入案例說明)
if04	Data Conversion 觀念總覽	資料轉換 / 搬遷情境 (舊系統上線初期匯入主檔 / 交易資料) 常見技術比較：手動 Open SQL (不建議・繞過業務邏輯檢查) / BDC (Batch Data Communication・模擬畫面操作・舊但穩定) / Direct Input (SAP 提供的特定物件專用高速轉檔程式) / BAPI-based conversion (目前主流・兼顧業務規則檢查與效能) ；搭配 LSMW 工具定位介紹 (GUI 操作工具・底層仍是呼叫上述技術之一) 。本題只講觀念對照・BDC 的實際動手寫程式留給 if05	承 if01~if03	已完成 (if04_data_conversion_overview.md ；四技術比較表 + 三個實務情境判斷・Direct Input 範例 RMDATIND/RFBIDE00/RFBIKR00 已用 sap-adt MCP 於 client 130 查證存在)
if05	BDC Program 實作	接續 if04 對 BDC 的概念介紹・這題實際動手寫一支 BDC 程式：SHDB 錄製一段畫面操作產生 BDC 程式骨架 (BDCDATA 內部表・PROGRAM / DYNPRO / DYNBEGIN / FNAM / FVAL 欄位意義) ；兩種執行方式對照——Call Transaction Method (CALL TRANSACTION ... USING it_bdcdata MODE 'N'/'A'/'E' UPDATE 'S'/'A'・即時執行・可直接用 MESSAGES INTO 收訊息) vs Session Method (BDC_OPEN_GROUP / BDC_INSERT / BDC_CLOSE_GROUP 建 Batch Input Session・SM35 背景執行・適合大量資料且不希望佔用前景作業時間) ；BDCMSGCOLL 錯誤訊息收集與轉譯成可讀文字 (FORMAT_MESSAGE / MESSAGE ID...TYPE...NUMBER...WITH) ；對	承 if04；對照 if01~if03 的 BAPI 呼叫方式	已完成 (if05_bdc_program.md ；已實際建立並啟用 ZCL_IF05_BDC_RUNNER・sap_run_unit_test 3/3 通過；format_messages 開發過程踩到「classic FM 的 VALUE() 參數在例外路徑仍會被清空」的真實 bug・已記錄；SHDB 錄製本身仍是 GUI-only、未做端對端測試)

#	主題	內容重點	銜接前面課程	狀態
		照 if01~if03 已教的 BAPI 呼叫方式，講清楚「什麼情境選 BDC (沒有對應 BAPI、要相容舊有畫面驗證邏輯)、什麼情境選 BAPI (有現成介面、要效能與結構化錯誤)」		
if06	RFC 基礎與 Native SQL	RFC (Remote Function Call) 機制：RFC-enabled FM 如何被外部系統 / 其他 SAP 系統呼叫；Native SQL (EXEC SQL...ENDEXEC) 語法與 Open SQL 的差異 (繞過資料庫無關層、直接下該資料庫原生語法、失去可移植性、僅用於特殊情境)；同一查詢的 Open SQL vs Native SQL 對照示範	對照 AMDP 課程「SQLScript 直接操作 HANA」的下推概念	已完成 (if06_rfc_native_sql.md ; ZR_IF06_NATIVE_SQL_DEMO 已建立、啟用、programrun 實測 Open SQL / Native SQL 兩段讀 SPFLI 結果一致；查證官方 ABAP 文件證實 Native SQL 新開發已凍結、SAP 建議新程式改用 if08 的 ADBC)
if07	DBCO 與 RFC Destination 設定	SM59 建立 RFC Destination (系統對系統呼叫的連線設定，含 Connection Type 分類)；DBCO 交易碼建立 Secondary Database Connection (讓 ABAP 程式連到非目前登入的資料庫)；兩者關係釐清：RFC Destination 是系統對系統呼叫，DBCO 是資料庫層連線，常被學員搞混	承 if06	已完成 (if07_dbco_rfc_destination.md ; 確認 ADT 無 SM59/DBCO API；用 datapreview/freestyle 讀到 RFCDES 真實 48 筆資料，DBCON 則被系統主動擋下讀取，兩者敏感度差異本身成為活教材)
if08	新的 JDBC 風格連線語法 (ADBC)	ABAP Database Connectivity (ADBC) 物件導向 API (CL_SQL_CONNECTION / CL_SQL_STATEMENT / CL_SQL_RESULT_SET) 取代舊式 EXEC SQL 的現代寫法；搭配 DBCO 設定的連線存取外部 / 異質資料庫；語法示範：取得連線 / 執行查詢 / 迴圈讀結果集 / CX_SQL_EXCEPTION 錯誤處理	承 if06/if07	已完成 (if08_adbc.md ; ZR_IF08_ADBC_DEMO 已建立、啟用、programrun 實測成功 / 失敗兩情境皆正確；意外發現並回頭修正 if07 一個誤解——DBCON 其實可被 Open SQL 正常讀取，if07 遇到的擋讀限制只在 datapreview/freestyle 工具層級)
if09	期末綜合實作	簡化版資料轉換程式：讀取來源資料 (內部表模擬匯入檔案)，依資料型態分兩種路徑處理——有對應 BAPI 的走逐筆呼叫標準 BAPI 寫入 + 收集 RETURN 訊息 + 統一 COMMIT / 失敗則 ROLLBACK；沒有對應 BAPI 的走 if05 教過的 BDC 方式，整合 if01~if05 全部技巧	呼應 REST rs10 批次處理模式、基礎課 ex23 LUW	已完成 (if09_capstone.md ; ZCL_IF09_CONVERSION_RUNNER + ZR_IF09_CAPSTONE_DEMO 已建立、啟用、sap_run_unit_test 3/3 通過，programrun 端對端驗證混合批次成功建立 2 筆真實訂位 (單號 00019802/00019803) 並正確路由 1 筆到 BDC；BDC 執行本身受 SHDB GUI-only 限制未做端對端)

if01 ~ if09 全數完成 (2026-07-21)，主課程結案。 SM59 / DBCO 已確認無 ADT API (見 if07/if08)；本課程沒有另外準備異質資料庫 / JDBC 連線標的，if08 的 ADBC 示範改用預設連線讀取 SCARR，足以驗證語法與錯誤處理，未涉及真正跨異質資料庫的連線測試——真的要示範這段，需要額外準備一個 DBCO 已設定好的異質資料庫連線，留待下一階段有實際需求時再補。下一階段主題未定案。